

Akshay N

Presenter Pack

Member A — Core Logic & Pathfinding

Slide 2

Maze Runner · Problem Statement 15

REVA University · Hackathon अभिनव · 07/10/2026

SECTION 1 — The project (everyone must know this)

What we built

A maze game where the player has to reach the exit while an enemy chases them using real pathfinding. The enemy recalculates the shortest route to the player on every single step, using either BFS or A*, and you can switch between the two live and watch the difference on screen.

From level 4 a **second enemy** joins running the *other* algorithm, so BFS and A* hunt the same player at the same time.

How it works, in 60 seconds

The maze is a grid of 1s and 0s — 1 is a wall, 0 is floor. The player and each enemy sit on one cell. Sixty times a second we read the keyboard, move everything one cell if its cooldown has expired, and redraw. The enemy decides where to go by running a search from itself to the player, then stepping onto the second cell of the route it found.

Everything else is detail on top of that.

The five facts everyone must be able to say

1. **BFS uses a FIFO queue, so it finds the shortest path.** It finishes every cell at distance d before touching anything at $d+1$, so the first time it reaches the goal it got there in the fewest steps.
2. **A* scores cells** $f = g + h$ — g is steps taken, h is Manhattan distance remaining. h must never overestimate or optimality is lost.
3. **A* expanded ~26% fewer cells but ran slower** in wall-clock time, because its heap costs $O(\log n)$ per operation against the deque's $O(1)$.
4. **The enemy used to camp the exit** — "farthest from the player" is the opposite corner, which is where the exit is. Fixed by maximising `min(distance to player, distance to exit)`.
5. **We tuned difficulty by measurement, not feel** — `autoplay.py` ran 500 bot playthroughs and found the enemy was only $1.1\times$ slower than the player.

The numbers

Tech	Python 3 + pygame-ce. No dataset, no network, no hardware
Tests	226 headless checks, all passing
Benchmark	50 mazes per configuration
Balance	500 bot playthroughs; L1–3 ~68–70%, L4–10 20–38%
Repo	github.com/Hyperval/maze-runner

Who built what

Person	Owns
Akhil Sathish Kumar	Maze generation, braiding, difficulty balance
Akshay N	Pathfinding — BFS, A*, distance maps
Abdullah Subhaan K	Rendering, menu, pause, HUD, test suite
Christen Mendis	benchmark.py, autoplay.py, docs, demo video

SECTION 2 — Your code: `pathfinding.py`

This is the file examiners press hardest on. You also wrote `scratch_bfs.py` — a from-memory rebuild of BFS — which is your proof you can explain it. Have it open.

`bfs(grid, start, goal)`

```
queue = deque([start])           # FIFO: first in, first out
seen = {start}                  # cells already queued, so we never revisit
came_from = {}                  # cell -> the cell we arrived from

while queue:
    current = queue.popleft()     # take the OLDEST cell
    if current == goal: ...done
    for nxt in neighbours(current):
        if nxt not in seen:
            seen.add(nxt)         # mark on ENQUEUE
            came_from[nxt] = current
            queue.append(nxt)
```

Why does this find the shortest path? Because it's a FIFO queue, every cell at distance 1 is processed before any at distance 2, which is processed before any at distance 3. So the first time you touch the goal, you got there in the fewest possible steps. That guarantee comes directly from the queue being first-in-first-out — swap it for a stack and you get DFS, which finds *a* path, not the shortest.

The detail worth volunteering: you mark cells as seen **when you enqueue them, not when you dequeue them**. Marking on dequeue lets the same cell get queued several times before it's processed — wasted work, and it can loop forever. You got this right in `scratch_bfs.py`; point at that line if asked.

Reconstructing the path: `came_from` is a breadcrumb trail. Start at the goal, follow the links back to the start, reverse the list.

```
astar(grid, start, goal)
```

Same idea, but a **priority queue** (heap) instead of a plain queue, always expanding the lowest-scoring cell:

```
f = g + h
g = steps actually taken to reach this cell
h = Manhattan distance remaining (|dx| + |dy|)
```

So A* leans toward the goal instead of spreading evenly in all directions.

"Why must `h` never overestimate?"

If it overestimated, A* could skip the cell that's actually on the shortest path because it looks worse than it really is, and you lose the optimality guarantee. Manhattan distance on a four-direction grid can never overestimate, because the real walk is at least that long and usually longer. A heuristic with that property is called **admissible**.

The `counter` in the heap entries. Entries are `(f, counter, cell)`. If two cells tie on `f`, Python compares the next item — the counter — so it never reaches the `cell` tuples, which would be an arbitrary comparison. Small detail, but if an examiner spots it and asks, that's the answer.

```
flood_distances(grid, start)
```

BFS with no goal — it keeps expanding until it has measured the whole reachable region, returning `{cell: distance}` for every cell.

Used for placing enemies and coins. **Why not Manhattan distance?** Because Manhattan ignores walls: it will call two cells "close" when a wall between them makes the real walk 40 steps.

Why the enemy steps to `path[1]`

```
self.cell = self.path[1]    # NOT path[0]
```

`path[0]` is the cell the enemy is already standing on. The next step is index 1.

"Why recompute every step?" The player keeps moving — a path computed once would be chasing where they *were*. At ~650 cells a search takes a fraction of a millisecond, so recomputing is cheap.

Complexity

$O(V + E)$, where V is cells and E the connections. On a grid each cell has at most 4 neighbours, so $E \approx 4V$, making it $O(\text{number of cells})$. Our maze is 31×21 up to 41×27 , so roughly 650–1100 cells.

SECTION 3 — Your presenting job

You present **Slide 2 — "You can see the AI thinking"**. Target: 60–75 seconds. This is the slide that earns Technical Implementation marks. Take your time on it; the others are shorter.

What to say

Point at the screenshot. There are two coloured regions on it.

This is one frame, level 5, with two enemies chasing the same player.

The purple region is every cell the BFS enemy expanded while searching — 175 cells. You can see it spreading out in all directions, because BFS has no idea where the player is until it reaches them.

The orange region is the A* enemy at the same instant — 100 cells. It leans toward the player, because every cell is scored by how far it's come plus an estimate of how far is left.

Both found a path of the same length. A* just looked at less of the maze to find it.

Then, if the live demo is up:

You can press TAB mid-chase and watch those counts change in the HUD.

The trap question — be ready for it

"So A* is better?"

Do **not** say yes. The honest answer scores much higher, and Christen's slide has the data:

It expands fewer cells, but it isn't always faster. We measured it — A* expanded about 26% fewer cells but actually ran *slower* in wall-clock time, because it uses a heap at $O(\log n)$ per operation while BFS uses a deque at $O(1)$. At our maze size the per-cell overhead outweighs the cells saved. On a much larger map, or where visiting a cell is expensive, A* wins.

That one answer demonstrates you understand the difference between an algorithm's complexity and its real-world constant factors. It is the strongest thing anyone on the team can say.

If they ask you to prove you wrote it

This is why `scratch_bfs.py` exists. Open it:

I rebuilt BFS from memory to make sure I understood it rather than just using it. It's covered by 16 tests that check it agrees with the main implementation on every maze.

Then walk them through the queue, the seen set, and the `came_from` dictionary.

Other questions that come to you

- "Why no diagonal movement?" — the walls are drawn on the grid, so a diagonal move would let the enemy slip through the corner point where two walls meet.
- "What if the enemy can't reach the player?" — can't happen, the generator guarantees every cell is connected and we test it across 20 mazes. But the code returns an empty path and the enemy simply doesn't move, rather than crashing.
- "Why BFS at all, if you have A*?" — BFS is simpler to implement and verify, and on an unweighted grid it's already optimal. We added A* to compare.

Final checklist

- ☐ Re-read `pathfinding.py` and `scratch_bfs.py` this morning
- ☐ Be able to write BFS on paper if asked — queue, seen set, came_from
- ☐ Rehearse the "so A* is better?" answer until it's automatic
- ☐ Know the two numbers on your slide: **175 and 100**